

Relazione di progetto

Tesori Nascosti è un gioco per due giocatori sviluppato in Python con interfaccia grafica realizzata tramite breezypythongui. Il gameplay si basa su round successivi in cui i giocatori costruiscono una mano di 5 carte, gestendo un budget di Punti Azione e sfruttando tre carte speciali con effetti strategici.

1. Strategia adottata

Obiettivo del sistema

L'applicazione deve permettere una partita completa a due giocatori su griglia 6×6, con turnazione, vincoli sulle azioni, calcolo punteggi in stile poker adattato e salvataggio persistente della classifica.

Architettura logica

La soluzione è organizzata secondo il pattern MVC (Model-View-Controller), separando la logica di dominio (Model) dalla gestione dell'interfaccia (View/Controller).

Modello dati

Rappresenta gli oggetti fondamentali del dominio, cioè Carta, Mazzo, Giocatore. Queste classi incapsulano stato e operazioni di base come copertura, mescolamento, gestione mano e punti azione.

Motore di valutazione

Un modulo dedicato alla valutazione della mano con regole di punteggio e gestione della Gemma come jolly vincolato. Il validatore fornisce metodi di check e una funzione di scoring che restituisce il punteggio del round come somma tra combinazione e punti azione residui.

Interfaccia e controllo del flusso

La GUI gestisce la griglia di pulsanti, i click, le azioni, l'alternanza dei turni, il timer, l'avanzamento round e la fine partita. Inoltre applica gli effetti delle carte speciali Moneta e Pergamena e coordina l'aggiornamento dello stato visuale, inclusa l'evidenza grafica di possesso delle carte in griglia.

Persistenza e classifica

Al termine della partita il risultato viene scritto su file di testo in append. La classifica viene ricostruita tramite parsing e ordinamento, rispettando le regole richieste, cioè punteggio vincitore decrescente, poi differenza punteggi, poi ordine alfabetico del vincitore.

2. Descrizione funzionale del gioco

Setup di un round

All'inizio di ogni round viene costruita una griglia 6×6 con 36 carte estratte da un mazzo più ampio. Le carte sono inizialmente coperte e vengono rivelate solo a seguito dell'azione del giocatore di turno.

Ogni giocatore parte con 15 Punti Azione e deve arrivare a una mano completa di 5 carte.

Turno di gioco e vincoli delle azioni

Durante il turno un giocatore seleziona una carta coperta della griglia per rivelarla temporaneamente, poi sceglie una tra le azioni consentite

Accetta

Costo 1 Punto Azione, aggiunge la carta rivelata alla mano solo se la mano non è completa.

Rifiuta

Costo 1 Punto Azione, la carta torna coperta e non viene assegnata. È consentita solo se i punti azione rimanenti sono strettamente maggiori del numero di carte mancanti per completare la mano.

Cambia

Costo 2 Punti Azione, sostituisce una carta in mano con la carta rivelata, scegliendo quale rimuovere. È consentita solo se in mano è presente almeno una carta e se i punti azione rimanenti rispettano il vincolo richiesto dopo l'azione.

La GUI implementa questi vincoli abilitando e disabilitando dinamicamente i pulsanti di azione in base allo stato corrente del turno, della mano e dei punti azione.

Carte speciali e loro effetti

Le carte speciali non appartengono a nessun seme e introducono meccaniche specifiche.

Gemma

Funziona come jolly solo per combinazioni basate su valori uguali, quindi coppia, doppia coppia, tris, full, poker. Non può completare scala, colore o scala reale. La mano che contiene Gemma viene valutata scegliendo il valore che massimizza il punteggio ammissibile.

Moneta

Quando viene accettata, assegna immediatamente un punto azione aggiuntivo. Non partecipa alle combinazioni ma può aumentare il punteggio finale in modo indiretto, dato che i punti azione residui vengono sommati al punteggio del round.

Pergamena

Quando viene accettata, consente di rivelare permanentemente una carta coperta a scelta

sulla griglia. La carta resta visibile per il resto del round e non viene assegnata automaticamente, quindi resta una informazione strategica.

Concludi e fine del round

Quando un giocatore completa la mano, può scegliere di concludere la propria partecipazione al round per conservare i punti azione residui. Il round termina quando entrambi i giocatori hanno concluso oppure quando entrambi hanno terminato i punti azione.

Al termine del round, per ciascun giocatore il punteggio viene calcolato sommando il punteggio della combinazione ai punti azione residui.

Il round successivo parte invertendo chi ha iniziato nel round precedente, così da bilanciare il vantaggio del primo turno.

Fine partita e criteri di vittoria

I punteggi dei round si accumulano e la partita termina quando almeno un giocatore raggiunge 110 punti. Se entrambi raggiungono la soglia nello stesso round vince chi ha il punteggio maggiore, poi a parità chi ha più punti azione residui, poi eventuale pareggio.

3. Gestione dell'audio di gioco e motivazioni della scelta tecnologica

Introduzione funzionale

Al fine di migliorare l'esperienza utente e rendere il gameplay più immersivo, il progetto integra una componente audio sotto forma di musica di sottofondo riprodotta in loop durante l'intera sessione di gioco. La musica non ha funzione ludica diretta, ma contribuisce a creare continuità, ritmo e coinvolgimento emotivo, elementi fondamentali nelle applicazioni interattive di tipo videoludico.

Scelta della libreria

Per la gestione dell'audio è stata utilizzata la libreria **pygame**, limitatamente al suo modulo di riproduzione sonora. La scelta è stata guidata da criteri di semplicità, affidabilità e compatibilità con l'architettura del progetto.

In particolare, pygame consente di:

- Riprodurre file audio in formato mp3 in modo diretto
- Gestire facilmente la riproduzione in loop continuo

- Avviare e mantenere l'audio in background senza interferire con il ciclo principale della GUI
- Operare in modo indipendente dalla logica di gioco e dal sistema di eventi grafici

L'uso di pygame è stato circoscritto esclusivamente alla gestione dell'audio, evitando dipendenze superflue o utilizzi non necessari della libreria. In questo modo, la componente sonora rimane modularizzata e non introduce complessità aggiuntiva nella struttura complessiva del software.

Integrazione con l'interfaccia grafica

La riproduzione musicale viene inizializzata una sola volta all'avvio dell'applicazione e rimane attiva per tutta la durata del gioco. Questa scelta progettuale consente di evitare chiamate ripetute o ridondanti al sistema audio durante i turni o le azioni dei giocatori, garantendo stabilità e continuità sonora.

Dal punto di vista architetturale, la musica non è legata a singole schermate o eventi, ma è trattata come una risorsa globale del gioco. Questo approccio rispetta il principio di separazione delle responsabilità, poiché la logica audio non interferisce con la gestione del turno, del punteggio o dell'interazione utente.

Impatto sulle prestazioni

Dal punto di vista della complessità algoritmica, la gestione dell'audio ha costo costante $O(1)$. La riproduzione in loop non comporta cicli computazionali dipendenti dalla dimensione dei dati di gioco e non influisce sulle prestazioni della GUI o sulla reattività dell'interfaccia.

L'inizializzazione del sistema audio avviene una sola volta e le operazioni successive sono delegate al motore interno della libreria, rendendo l'impatto computazionale trascurabile rispetto alle operazioni di aggiornamento della griglia o di valutazione delle mani di gioco.

Coerenza progettuale

L'integrazione dell'audio tramite pygame risulta coerente con l'obiettivo del progetto, che è quello di realizzare un gioco completo, interattivo e usabile. La scelta tecnologica privilegia una soluzione consolidata e leggera, mantenendo il codice leggibile, separato per responsabilità e facilmente estendibile in futuri sviluppi, come l'aggiunta di effetti sonori legati a specifiche azioni di gioco.

4. Classi e metodi principali

4.1 Carta

La classe Carta rappresenta sia carte numeriche con valore e seme, sia carte speciali con tipoSpeciale. Gestisce inoltre lo stato di visibilità

- **Attributi principali**
valore, seme, tipoSpeciale
coperta
rivelataPermanente
- **str**
Fornisce una rappresentazione testuale della carta, distinguendo speciali e numeriche, con gestione delle figure per la sola resa testuale. Questo metodo è utile per debug e per eventuali elementi testuali in GUI.
- **gira_carta**
Inverte lo stato (da coperta a scoperta e viceversa), ma blocca l'operazione se la carta è stata rivelata permanentemente tramite pergamena.
- **rivela_permanente**
Imposta rivelataPermanente e scopre la carta, rendendo la scelta della pergamena persistente per il round.
- **reset**
Ripristina lo stato iniziale. È usato quando una carta viene rimossa dalla mano o quando si avvia un nuovo round.

Complessità

Tutte le operazioni della classe Carta sono $O(1)$, perché agiscono su un numero costante di attributi.

4.2 Mazzo

La classe Mazzo costruisce l'insieme di carte, mescola e fornisce le 36 carte per la griglia 6×6 .

- **creaMazzo**
Genera le carte numeriche per ogni seme e aggiunge le carte speciali, poi mescola.
- **mescola**
Mescolamento casuale della lista tramite shuffle.
- **estrai_36**
Restituisce le prime 36 carte e le rimuove dal mazzo, per popolare la griglia del round.

Complessità

- creaMazzo è $O(N)$ dove N è il numero di carte generate, perché crea tutte le istanze e poi mescola
 - mescola è $O(N)$ nel caso medio, coerente con l'algoritmo di shuffle su lista
 - estrai_36 è $O(36)$ per l'estrazione logica, che in pratica è costante, e l'operazione di slicing è proporzionale alla dimensione estratta
-

4.3 Giocatore

La classe Giocatore incapsula nome, mano, punti azione e stato di conclusione del round.

- **aggiungi_carta**
Aggiunge una carta alla mano se non è piena, cioè massimo 5 carte.
- **rimuovi_carta**
Rimuove una carta dalla mano e invoca reset sulla carta rimossa, così da ripristinarne lo stato.
- **reset_round**
Resetta la mano, ripristina 15 punti azione e imposta concluso a falso.

Complessità

- `aggiungi_carta` è $O(1)$
 - `rimuovi_carta` è $O(K)$ dove K è la dimensione della mano, al massimo 5, quindi sostanzialmente costante
 - `reset_round` è $O(K)$ con K al massimo 5
-

4.4 Validator

Questa classe implementa le regole di valutazione del poker adattato e il calcolo del punteggio round, includendo la gestione della Gemma.

- **PUNTEGGI**
Mappa combinazione a punteggio fisso, con scala reale 100, poker 50, full 30, colore 25, scala 20, tris 15, doppia coppia 10, coppia 5.
- **valutaMano(carte, puntiAzioneResidui)**
Calcola il punteggio della combinazione e somma i punti azione residui, come richiesto dalla consegna.
- **calcolaPunteggioCombo**
Valuta separatamente scale e colori, poi gruppi di valori uguali, e prende il massimo tra i due rami, garantendo che la migliore combinazione valida determini il punteggio combo.
- **gestisciGemma**
Se la mano contiene Gemma, la tratta come jolly senza seme che può assumere un valore numerico per migliorare solo combinazioni a gruppi. Implementa una ricerca sul valore da assegnare basata sulle occorrenze, scegliendo l'esito con punteggio massimo tra poker, full, tris, doppia coppia, coppia.
- **checkScalaReale, checkPoker, checkFull, ecc**
Metodi di controllo che verificano le condizioni delle combinazioni, usando conteggi di valori e verifiche su semi e consecutività.

Complessità

Sia per vincolo di gioco sia per scelta progettuale, la mano ha cardinalità massima 5, quindi i costi reali sono costanti. In termini asintotici generali

- estrazione valori, conteggi e check basati su Counter sono $O(K)$
 - ordinamento valori per la scala è $O(K \log K)$
 - `gestisciGemma` itera sui valori distinti presenti nella mano, al massimo K , quindi $O(K)$ con operazioni interne costanti
- Con K fissato a 5, l'esecuzione è sempre molto efficiente e adatta a aggiornamenti frequenti in GUI, anche durante il turno.
-

4.5 Gioco e GUI

La classe principale della GUI gestisce lo stato globale della partita, la griglia, i pulsanti delle azioni, il timer e l'alternanza dei turni.

- **gestisciTurno**
Aggiorna pannello informativo e abilita/disabilita i controlli coerentemente con vincoli di punti azione e dimensione mano. Aggiorna anche un punteggio provvisorio come somma tra punti totalizzati e valutazione corrente della mano con punti azione residui.
- **cambiaTurno**
Alterna il giocatore attivo, gestisce i casi in cui un giocatore ha concluso o ha esaurito i punti azione, e aggiorna la griglia mostrando le carte assegnate solo al proprietario, mentre le altre restano coperte, salvo rivelazioni permanenti da pergamena.
- **rivelaCarta**
Gestisce il click sulla griglia. Include due sottodinamiche importanti
modalità pergamena, in cui una carta non assegnata viene rivelata permanentemente
modalità scambio, in cui si seleziona quale carta sostituire nella mano durante l'azione Cambia
- **azioneConcludi**
Permette di chiudere il round per il giocatore quando la mano è completa, come richiesto dalla consegna.
- **fineRound**
Arresta temporaneamente il timer, calcola e accumula i punteggi dei giocatori chiamando il Validator, ripristina la griglia e prepara il round successivo se nessuno ha raggiunto 110 punti.
- **finePartita**
Applica le regole di vittoria e spareggio, determinando vincitore o pareggio in modo coerente con la consegna.

Complessità

Molte operazioni della GUI iterano sulla griglia, quindi hanno costo $O(G)$ con G pari a 36, che è costante nel gioco. L'aggiornamento del turno, inclusa la gestione delle immagini e degli stati dei pulsanti, è quindi efficiente e compatibile con aggiornamenti frequenti.

Avvio dell'applicazione, Menu principale e Classifica

Ruolo della classe main.py nel progetto

Il file main.py rappresenta il punto di ingresso dell'intera applicazione.

La sua funzione è quella di inizializzare l'ambiente grafico, gestire le schermate preliminari e di servizio e avviare il gioco vero e proprio.

Dal punto di vista architetturale, questa classe non implementa logiche di gioco o algoritmi di punteggio, ma svolge un ruolo di controller di alto livello, occupandosi di:

- presentare il menu principale
- gestire la riproduzione della musica di sottofondo
- avviare la finestra di gioco
- visualizzare la classifica persistente
- coordinare la transizione tra le diverse finestre dell'applicazione

Questo approccio consente di mantenere separata la logica di avvio e navigazione dalla logica interna del gioco, migliorando leggibilità, modularità e manutenibilità del codice.

Integrazione di Tkinter e limiti di BreezyPythonGUI nella gestione della Classifica

Per la realizzazione della schermata di classifica è stato necessario integrare l'uso diretto della libreria standard tkinter, affiancandola al wrapper breezypythongui utilizzato per il core del gioco. La scelta è motivata dai vincoli di personalizzazione estetica e funzionale imposti dal wrapper, che astrae la gestione dei widget semplificandone il layout a griglia ma riducendo drasticamente il controllo sui parametri di rendering a basso livello.

Nello specifico, la schermata di classifica richiedeva requisiti non soddisfacibili tramite le classi standard di breezypythongui:

- **Sovrapposizione di elementi (Layering):** La necessità di posizionare una Listbox scorrevole al di sopra di un'immagine di sfondo personalizzata ha richiesto l'uso del widget Canvas e del metodo create_window, funzionalità gestite in modo nativo da tkinter ma non esposte chiaramente dal wrapper.
- **Styling avanzato dei Widget:** Per ottenere una resa grafica coerente con il tema del gioco, è stato necessario accedere a parametri specifici del widget Listbox quali borderwidth e highlightthickness. L'impostazione di questi valori a zero, fondamentale per eliminare i bordi di default e il rettangolo di focus del sistema operativo, non è

supportata in modo trasparente da breezypythongui, che tende a forzare stili predefiniti.

L'adozione di un approccio ibrido ha permesso di mantenere la rapidità di sviluppo per la griglia di gioco (dove breezypythongui è efficiente) garantendo al contempo la flessibilità necessaria per curare l'esperienza visiva nelle schermate di menu e classifica.

Classe MenuPrincipale

Responsabilità principali

La classe MenuPrincipale, derivata da EasyFrame, rappresenta la schermata iniziale dell'applicazione.

Essa ha il compito di offrire un punto di accesso chiaro e immediato al gioco, combinando elementi grafici, interazione utente e componente audio.

In particolare, la classe gestisce:

- l'apertura dell'applicazione a schermo intero
- il caricamento e il ridimensionamento dinamico dell'immagine di copertina
- l'inizializzazione e la riproduzione della musica di sottofondo in loop
- la visualizzazione del pulsante di avvio del gioco

Inizializzazione e layout grafico

Durante l'inizializzazione, la finestra viene adattata automaticamente alla risoluzione dello schermo dell'utente. L'immagine di copertina viene ridimensionata mantenendo un'elevata qualità visiva, garantendo una resa uniforme su dispositivi con risoluzioni differenti.

L'uso di un canvas permette di sovrapporre elementi interattivi all'immagine di sfondo, consentendo il posizionamento del pulsante di avvio in modo flessibile e centrato rispetto allo schermo.

Gestione dell'audio

All'avvio del menu viene inizializzata la musica di sottofondo, che viene riprodotta in loop continuo.

La componente audio è indipendente dalla logica di gioco e viene avviata una sola volta, evitando duplicazioni o interferenze con il ciclo di eventi grafici.

Nel caso in cui la riproduzione audio non sia disponibile, l'applicazione continua comunque il proprio funzionamento, preservando la robustezza dell'avvio.

Avvio del gioco

Il metodo `avvia_gioco` gestisce la transizione dal menu alla finestra principale della partita. Alla pressione del pulsante "Gioca", la finestra del menu viene chiusa e viene istanziata la classe che gestisce il gioco vero e proprio.

Questa separazione garantisce che il menu resti una schermata indipendente e non interferisca con la gestione dei turni, della griglia o del punteggio.

Complessità computazionale

Le operazioni svolte da `MenuPrincipale` hanno costo computazionale costante:

- caricamento immagine e inizializzazione audio sono eseguiti una sola volta
- la gestione del pulsante e dell'evento di click è $O(1)$
- non sono presenti cicli dipendenti dalla dimensione dei dati di gioco

Pertanto, la complessità algoritmica della classe è $O(1)$ rispetto alle dimensioni del problema, con un impatto trascurabile sulle prestazioni complessive dell'applicazione.

Classe Classifica

Funzione della schermata di classifica

La classe `Classifica`, anch'essa derivata da `EasyFrame`, è dedicata alla visualizzazione dei risultati delle partite precedenti salvati in modo persistente.

Essa fornisce una schermata separata, accessibile dall'interfaccia, che consente di consultare la classifica in forma testuale, ordinata secondo le regole definite nel modulo di gestione dei dati.

Caricamento e visualizzazione dei dati

La classe:

- carica uno sfondo grafico dedicato
- legge i dati della classifica da file di testo tramite la classe `GestoreClassifica`

- inserisce le righe formattate all'interno di una Listbox

La scelta di una Listbox consente una visualizzazione ordinata, leggibile e facilmente scorribile dei risultati, mantenendo uno stile coerente con l'estetica del gioco.

Navigazione

Un pulsante consente di tornare al menu principale, chiudendo la finestra della classifica e riaprendo la schermata iniziale.

Questo meccanismo garantisce una navigazione chiara e lineare tra le schermate dell'applicazione.

Complessità computazionale

Sia il caricamento dello sfondo sia la creazione dell'interfaccia grafica hanno costo costante.

La lettura e visualizzazione della classifica dipendono dal numero di righe salvate, indicato con R .

- lettura e parsing dei dati: $O(R)$
- inserimento delle righe nella Listbox: $O(R)$

La complessità complessiva della schermata di classifica è quindi **$O(R)$** , risultando adeguata e scalabile rispetto al numero di partite salvate.

Avvio dell'applicazione

Il blocco finale del file main.py avvia l'applicazione istanziando la classe MenuPrincipale e avviando il ciclo principale dell'interfaccia grafica.

Questo rende main.py il punto di ingresso formale del progetto, in conformità con le convenzioni standard per le applicazioni Python dotate di interfaccia grafica.

4.6 Gestore Classifica

La classe GestoreClassifica gestisce la persistenza dei dati di gioco su file system. Oltre alle operazioni di I/O, questo modulo è responsabile della normalizzazione e della validazione dei dati storici per garantire la coerenza della classifica anche in presenza di formati di salvataggio precedenti o casi di pareggio.

Attributi e Costanti

- **VALORE_PAREGGIO**: Costante stringa ("PAREGGIO") utilizzata come valore standardizzato nel campo vincitore quando non vi è un singolo trionfatore.
- **Partita** : Rappresenta il record di una singola partita. Include metodi statici per la conversione da/a stringa e la pulizia dei dati.

Normalizzazione e Retrocompatibilità

La classe implementa una logica robusta per la gestione del campo vincitore tramite il metodo statico `_normalizza_vincitore`. Questo metodo verifica che il nome del vincitore coincida con uno dei due giocatori; in caso contrario (o se il campo è vuoto/nullo), il valore viene forzato alla costante `VALORE_PAREGGIO`.

Il parsing delle righe (`from_line`) gestisce inoltre la retrocompatibilità: se il file contiene righe generate da versioni precedenti del software (con 9 campi anziché 10), il sistema le accetta automaticamente assegnando il valore di default per il pareggio, prevenendo errori di lettura.

- **registra_partita**: Metodo di scrittura in append su file `classifica.txt`. Prima del salvataggio, invoca la normalizzazione del vincitore per assicurare che nessun dato "sporco" o incoerente venga scritto su disco. Salva timestamp ISO, durata, metriche di gioco e vincitore separati da pipe (`|`).
- **classifica_come_testo**: Il metodo ricostruisce le statistiche leggendo l'intero storico. Per ogni giocatore calcola:
 - **Vittorie**: Incrementate solo se il campo *vincitore* normalizzato coincide strettamente con il nome del giocatore. Le partite terminate in pareggio vengono conteggiate nel numero totale di partite giocate ma non incrementano il contatore delle vittorie per nessuno dei due contendenti.
 - **Statistiche**: Calcola miglior punteggio (*best*), media punti e durata media della partita.

L'ordinamento finale utilizza un criterio stabile multi-chiave:

1. Numero di Vittorie (decrescente).
2. Miglior Punteggio (decrescente).
3. Media Punteggio (decrescente).
4. Nome (crescente alfabetico).

Complessità

Le operazioni mantengono efficienza elevata: $O(1)$ per la scrittura normalizzata e $O(R)$ per la lettura e l'adattamento delle righe legacy, dove R è il numero di partite storiche.

5. Persistenza e ordinamento della classifica

Salvataggio

Le partite vengono salvate su file di testo in modalità append. Ogni riga rappresenta una singola partita conclusa e contiene informazioni quali data, durata della partita, numero di round, nomi dei giocatori, punteggi finali, punti azione residui e nominativo del vincitore.

I campi sono separati da un carattere delimitatore, così da consentire un parsing semplice e deterministico in fase di lettura.

Questa soluzione garantisce una persistenza leggera, facilmente ispezionabile e indipendente da sistemi di database esterni.

Ordinamento

La classifica viene ordinata secondo i criteri definiti dalla consegna:

- punteggio del vincitore in ordine decrescente
- a parità di punteggio, differenza tra i punteggi dei due giocatori in ordine decrescente
- a ulteriore parità, ordine alfabetico crescente del nome del vincitore

L'ordinamento è applicato in memoria sui dati letti dal file, prima della visualizzazione all'utente.

Algoritmo di ordinamento adottato

Per l'ordinamento della classifica è stato implementato esplicitamente l'algoritmo Insertion Sort.

Si tratta di un algoritmo di ordinamento comparativo semplice, che costruisce progressivamente la sequenza ordinata inserendo ogni nuovo elemento nella posizione corretta rispetto a quelli già elaborati.

La scelta di Insertion Sort è coerente con il contesto applicativo del progetto, poiché:

- il numero di risultati salvati è limitato e cresce lentamente
- l'algoritmo è semplice da implementare e da verificare
- il comportamento è deterministico e facilmente controllabile
- consente di applicare in modo chiaro un criterio di ordinamento multi-livello

L'ordinamento multi-chiave viene gestito direttamente durante il confronto tra due elementi, valutando in sequenza il punteggio del vincitore, la differenza dei punteggi e infine il nome, garantendo il rispetto rigoroso delle regole di classifica.

Complessità computazionale

Assumendo R risultati salvati:

- la lettura e il parsing del file hanno complessità $O(R)$
- l'algoritmo Insertion Sort ha complessità $O(R^2)$ nel caso peggiore

Nel contesto dell'applicazione, il valore di R rimane contenuto, rendendo il costo computazionale dell'ordinamento pienamente accettabile.

La semplicità dell'algoritmo e la chiarezza del processo di ordinamento risultano quindi più rilevanti della scalabilità asintotica, senza impatti percepibili sulle prestazioni o sull'esperienza utente.

6. Ruolo dei componenti del gruppo e organizzazione del lavoro

L'organizzazione del lavoro ha seguito una suddivisione per responsabilità, così da garantire parallelizzazione delle attività e un'integrazione pulita dei diversi componenti del sistema.

Nel corso del progetto è stato utilizzato in modo intensivo un sistema di versionamento basato su Git, con repository ospitato su GitHub. Ogni membro del gruppo ha creato e gestito autonomamente il proprio branch di lavoro, effettuando commit frequenti, generalmente quotidiani e spesso più volte al giorno, in modo da documentare in maniera incrementale l'avanzamento delle attività.

Questa metodologia ha consentito di mantenere il codice sempre coerente, di individuare rapidamente eventuali problemi e di integrare progressivamente le diverse parti del progetto. Al termine dello sviluppo, il lavoro dei singoli branch è stato integrato attraverso un merge finale nel branch principale, ottenendo una versione stabile e completa dell'applicazione.

Un **commit** rappresenta una registrazione puntuale dello stato del codice in un determinato momento, accompagnata da una descrizione delle modifiche effettuate. Attraverso i commit è possibile tracciare l'evoluzione del progetto nel tempo e ripristinare versioni precedenti in caso di necessità.

Un **branch** è una linea di sviluppo separata che consente di lavorare su una funzionalità o su un insieme di modifiche in modo isolato rispetto al codice principale. L'utilizzo dei branch permette a più sviluppatori di lavorare in parallelo senza interferenze dirette.

Metodologia di sviluppo e Testing: Il ciclo di vita del progetto è stato articolato in fasi distinte per garantire efficienza e qualità del software finale. La prima fase, di analisi e pianificazione, è stata dedicata alla definizione dell'architettura del sistema e alla scomposizione del problema in moduli indipendenti (Model, View, Controller).

In questa sede sono stati assegnati i compiti specifici ai membri del team, stabilendo a priori le interfacce di comunicazione tra le classi per facilitare l'integrazione successiva.

A seguito della fase di implementazione, il progetto è stato sottoposto a un rigoroso processo di validazione. Oltre al testing unitario svolto autonomamente dai singoli sviluppatori durante la stesura del codice, è stata condotta una sessione di stress testing distribuendo una versione preliminare del gioco a un gruppo di utenti esterni (conoscenti e tester volontari). Questa fase di beta testing si è rivelata fondamentale per due scopi principali:

- **Rilevamento Bug:** Identificare anomalie ed "edge cases" (casi limite) che non erano emersi durante le simulazioni interne degli sviluppatori.
- **Feedback sull'esperienza utente:** Raccogliere pareri oggettivi sulla chiarezza dell'interfaccia e sulla fluidità del gameplay, permettendo di apportare le rifiniture necessarie prima del rilascio finale.

Contributo dei singoli componenti

- **Riccardo Sciabbarrasi** ha curato il modello dati e la persistenza, progettando le strutture di base del gioco e le operazioni di gestione dello stato degli oggetti, nonché i meccanismi di salvataggio e lettura dei risultati.
 - **Andrea Barilà** ha curato il motore logico di valutazione della mano, con particolare attenzione alla gestione vincolata della carta speciale Gemma e alla corretta attribuzione dei punteggi secondo la tabella prevista. Inoltre, ha contribuito allo sviluppo della logica di memorizzazione e ordinamento della classifica.
 - **Marco Oliveri** ha curato l'interfaccia grafica e il controllo del flusso di gioco, gestendo turni, eventi di click, abilitazione delle azioni, round, timer e applicazione degli effetti delle carte speciali Moneta e Pergamena. Inoltre, ha sviluppato l'interfaccia del menu principale, la schermata della classifica e la logica di ricostruzione visiva dei risultati salvati.
 -
-